

UNIVERSIDADE FEDERAL DE GOIÁS

Curso de Ciência da Computação

Disciplina: Compiladores

Prof.: Sérgio Francisco da Silva

Primeira Lista de Exercícios de Compiladores

Exercício 00 – Explique a diferença funcional entre um compilador e um interpretador.

Exercício 01 – Quais são as etapas de um compilador? Explique sucintamente cada uma delas.

Exercício 02 – Em que consiste a análise léxica? Exemplifique.

Exercício 03 – Por que o analisador léxico repassa ao analisador sintático os *tokens* e não os lexemas lidos do arquivo-fonte?

Exercício 04 – Como o analisador léxico diferencia uma palavra reservada (ex.: `for`) de um identificador (ex.: `cont`)?

Exercício 05 – Que tipos de erros podem ser detectados pelo analisador léxico? Dê exemplos.

Exercício 06 – Quais as funções da análise sintática?

Exercício 07 – Explique qual a relação e para que servem expressões regulares, autômatos finitos, classe de *tokens* e lexemas.

Exercício 08 – Construa um autômato finito para aceitar as palavras reservadas `case`, `char`, `const` e `continue`.

Exercício 09 – Considere as classes de *tokens* (ou simplesmente *tokens*):

Números inteiros não sinalizados

Números reais não sinalizados (sem uso de potências de 10 ou notação científica)

Operadores aritméticos: `+`, `-`, `/`, `*`, `(`, `)`

Operador de atribuição: `=`

Identificadores

Escreva expressões regulares para representar e construa um autômato finito para reconhecer as classes de *tokens* acima.

Exercício 10 – A maioria das linguagens diferencia maiúsculas de minúsculas, de modo que as palavras-chave podem ser escritas apenas de uma maneira, e as expressões regulares descrevendo seu lexema são muito simples. Entretanto algumas linguagens como SQL, não

fazem tal diferenciação, de modo que uma palavra-chave pode ser escrita em minúsculas, ou em maiúsculas ou em qualquer mistura de maiúsculas e minúsculas. Assim, a palavra-chave SELECT de SQL também pode ser escrita como `select`, `Select` ou `sELeCT`, por exemplo. Mostre como escrever uma expressão regular para palavras-chave em uma linguagem que não diferencie maiúsculas de minúsculas. Ilustre a ideia escrevendo uma expressão regular para “select” em SQL.

Exercício 11 – Escreva uma expressão regular para comentários consistindo em uma cadeia cercada por /* e */, sem um */ intercalado, a menos que esteja dentro de aspas (“).

Exercício 12 – Converta a expressão regular do **Exercício 11** para um autômato finito não determinístico com transições vazias usando a construção de Thompson.

Exercício 13 – Converta o autômato finito não determinístico com ϵ -transições do **Exercício 12** para um autômato finito determinístico.

Exercício 14 – Dê gramáticas livres de contexto para as seguintes linguagens:

1. O conjunto de todas as cadeias de 0s e 1s tais que cada 0 seja imediatamente seguido por pelo menos um 1.
2. O conjunto de todas as cadeias de 0s e 1s que sejam palíndromos, ou seja, a cadeia pode ser lida da mesma forma de trás para frente.
3. O conjunto de todas as cadeias de 0s e 1s com um número igual de 0s e 1s;

Exercício 15 – Dada a gramática:

$$A \rightarrow A A \mid (A) \mid \epsilon$$

Descreva a linguagem que ela gera, e mostre que ela é ambígua.

Exercício 16 – Dada a gramática:

$$E \rightarrow E + T \mid E - T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid \text{num}$$

Escreva derivações mais à direita e árvores sintáticas para as expressões a seguir:

$$3+4*5-6$$

$$3*(4-5+6)$$

$$3-(4+5*6)$$

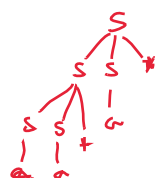
$$9$$

Exercício 17 – Considere a gramática livre de contexto:

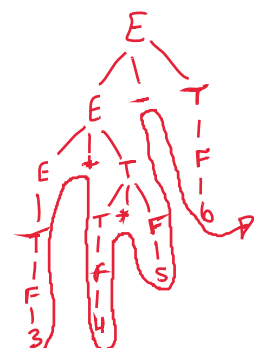
$$S \rightarrow S S + \mid S S * \mid a$$

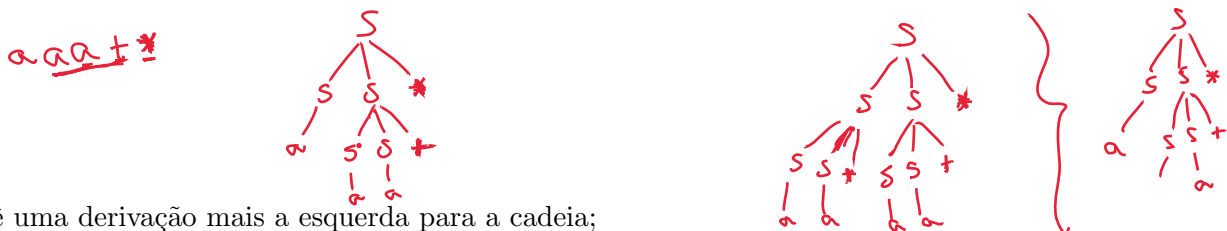
e a cadeia $aa + a*$.

$$aa * aa + *$$



$$3+4*5-6$$





1. Dê uma derivação mais a esquerda para a cadeia;
2. Dê uma derivação mais a direita para a cadeia;
3. Dê uma árvore de derivação para a cadeia;
4. A gramática é ambígua ou não? Justifique sua resposta.
5. Descreva a linguagem gerada por essa gramática.

Não é ambígua. Não é possível gerar duas ou mais árvores de derivação para qualquer cadeia que pertence a linguagem.

*expressões na notação pós-fixa envolvendo os operadores + e * e a como operandos.*
Exercício 18 – Repita o **Exercício 18** para cada uma das seguintes gramáticas e cadeias.

1. $S \rightarrow 0S1|01$ e a cadeia 000111.
2. $S \rightarrow +SS|*SS|a$ e a cadeia $+*aaa$.
3. $S \rightarrow S(S)S|\epsilon$ com cadeia $((())())$.
4. $S \rightarrow S + S|SS|S * S|(S)|S|a$ com a cadeia $(a + a) * a$.
5. $S \rightarrow (L)|a$ e $L \rightarrow L, S|S$ e a cadeia $((a, a), a, (a))$.
6. $S \rightarrow aSbS|bSaS|\epsilon$ e a cadeia $aabbab$
7. a gramática a seguir para expressões booleanas:
 $bexpr \rightarrow bexpr \text{ or } bterm \mid bterm$
 $bterm \rightarrow bterm \text{ and } bfactor \mid bfactor$
 $bfactor \rightarrow \text{not } bfactor \mid (bexpr) \mid \text{true} \mid \text{false}$

Exercício 19 – Um símbolo X (terminal ou não) de uma gramática é inútil se não houver derivação da forma $S \xRightarrow{*} wXy \xRightarrow{*} wxy$. Ou seja, X nunca aparecerá na derivação de alguma sentença.

1. Dê um algoritmo para eliminar todas as produções contendo símbolos inúteis de uma gramática .
2. Aplique seu algoritmo à gramática:
 $S \rightarrow 0 \mid A$
 $A \rightarrow A B$
 $B \rightarrow 1$

Exercício 20 – A gramática a seguir define expressões regulares sob os símbolos **a** e **b**, usando o operador $+$ no lugar do operador $|$ para a união, a fim de evitar conflito com a barra vertical usada como meta-símbolo nas gramáticas:

$rexpr \rightarrow rterm + rexpr \mid rterm$
 $rterm \rightarrow rfactor rterm \mid rfactor$
 $rfactor \rightarrow rfactor^* \mid rprimary$
 $rprimary \rightarrow a|b$

Fatore esta gramática à esquerda para eliminar o problema de prefixos comuns.

Exercício 21 – Para cada uma das gramáticas a seguir, elimine o problema de prefixos comuns e de recursão à esquerda, caso estes problemas existam. Em seguida, para cada item, construa a tabela de análise sintática LL(1) e mostre a aplicação desta para a *string* dada.

1. $S \rightarrow 0S1|01$, com a *string* 000111
2. $S \rightarrow +SS|*SS|a$, com a *string* + * aaa
3. $S \rightarrow S(S)S|\epsilon$, com a *string* (()())
4. $S \rightarrow S + S|SS|(S)|S * |a$, com a *string* (a + a) * a
5. $S \rightarrow (L)|a$ e $L \rightarrow L, S|S$, com a *string* ((a, a), a, (a))

Exercício 22 – Para cada uma das gramáticas a seguir, elimine o problema de prefixos comuns e de recursão à esquerda, caso estes problemas existam.

1. $S \rightarrow 0S1|01$
2. $S \rightarrow S(S)S|\epsilon$
3. $S \rightarrow S + S|SS|(S)|S * |a$
4. $S \rightarrow (L)|a$ e $L \rightarrow L, S|S$

Exercício 23 – Para cada uma das gramáticas abaixo, construa o autômato LR(0).

1. $S \rightarrow 0S1|01$.
2. $S \rightarrow +SS|*SS|a$.
3. $S \rightarrow S(S)S|\epsilon$.
4. $S \rightarrow S + S|SS|S * S|(S)|S|a$.
5. $S \rightarrow (L)|a$ e $L \rightarrow L, S|S$.
6. $S \rightarrow aSbS|bSaS|\epsilon$.
7. a gramática a seguir para expressões booleanas:
 $bexpr \rightarrow bexpr \text{ or } bterm \mid bterm$
 $bterm \rightarrow bterm \text{ and } bfactor \mid bfactor$
 $bfactor \rightarrow \text{not } bfactor \mid (bexpr) \mid \text{true} \mid \text{false}$

Lembre de estudar também o material dado em sala de aula.